



Chapter 3: Fundamental Network Modelling Techniques — Graphs

Comprehensive Study Guide — MOBAN 2025-2026

Prof. Mario Pickavet — IDLab, Dept. of Information Technology

Table of Contents

1. [Why Graph Theory in MOBAN?](#)
2. [Definitions and Notations](#)
 - 2.1 Undirected Graphs
 - 2.2 Directed Graphs (Digraphs)
 - 2.3 Paths, Walks, and Cycles
 - 2.4 Cuts and Connectivity
3. [Minimum Spanning Tree](#)
4. [Shortest Path Problem](#)
 - 4.1 Unweighted: Moore Algorithm
 - 4.2 Weighted (Positive): Dijkstra
 - 4.3 Weighted (General): Ford-Bellman-Moore
 - 4.4 All-Pairs: Floyd
5. [Maximum Flow Problem](#)
 - 5.1 Network Flow Model
 - 5.2 Residual Network Concept
 - 5.3 Flow Augmenting Path Algorithm
 - 5.4 Max-Flow Min-Cut Theorem
 - 5.5 Connectivity via Max Flow
6. [Minimum Cost Flow Problem](#)
 - 6.1 Problem Definition
 - 6.2 Residual Network for Min Cost

- 6.3 Busacker-Gowen Algorithm
 - 6.4 Multiple Supply/Demand Vertices
7. [Multi-Commodity Flow Problems](#)
 8. [Network Availability](#)
 9. [Algorithm Comparison Summary](#)
 10. [Exercise Problems Overview](#)

1. Why Graph Theory in MOBAN?

Communication network problems fundamentally reduce to graph problems:

Network Question	Graph Problem
Best route from A to B (cheapest, fastest, most reliable)	Shortest path problem
Maximum data rate between server and client	Maximum flow problem
Cheapest way to route a given demand	Minimum cost flow problem
Connect all nodes at minimum infrastructure cost	Minimum spanning tree

Abstract modelling of a communication network requires representing:

- **Topology** — which nodes are connected by which links
- **Transfer of information** — flows through the network
- **Parameters** — link costs, capacities, delays
- Cost and capacity of equipment

Graph algorithms are also used beyond networking: GPS route planning, logistics, electricity networks, sewer systems, oil distribution.

2. Definitions and Notations

2.1 Undirected Graph

A graph $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ consists of:

- $\mathbf{V}$ = vertex set (nodes; e.g., routers, base stations)
- $\mathbf{E}$ = edge set (links; e.g., fiber, wireless link)
- Each edge $\mathbf{e} = \{\mathbf{u}, \mathbf{v}\}$ is an *unordered* pair of vertices

Key quantities:

Term	Symbol	Meaning
Graph order	$p = \mathbf{V} $	Number of vertices
Graph size	$q = \mathbf{E} $	Number of edges
Incidence $I(v)$	—	Set of edges incident to vertex v
Adjacency $A(v)$	—	Set of vertices adjacent to v
Degree $\delta(v)$	—	Number of edges incident to v

Subgraph $\mathbf{G}[W]$: Induced by a subset $W \subseteq V$ — takes all vertices in W plus all edges whose *both* endpoints are in W .

Assumption: No parallel edges (at most one edge between any two vertices), no loop edges (no edge from a vertex to itself).

2.2 Directed Graph (Digraph)

A digraph $\mathbf{D} = (\mathbf{V}, \mathbf{A})$ consists of:

- $\mathbf{V}$ = vertex set
- $\mathbf{A}$ = arc set; each arc $\mathbf{a} = (\mathbf{u}, \mathbf{v})$ is an *ordered* pair

Term	Meaning
Tail of arc (u,v)	vertex u (from which the arc departs)
Head of arc (u,v)	vertex v (to which the arc points)
From-incidence $I(v)$	Arcs incident <i>from</i> v
To-incidence $I'(v)$	Arcs incident <i>to</i> v
Out-degree $\delta(v)$	Number of arcs leaving v
In-degree $\delta'(v)$	Number of arcs entering v

Anti-parallel arcs are **allowed**: arc (u,v) and arc (v,u) can both exist.

Strong vs Weak connectedness:

Type	Definition
Strongly connected	For every pair (u, v) : both a $(u \rightarrow v)$ -path AND a $(v \rightarrow u)$ -path exist
Weakly connected	For every pair (u, v) : a $(u \rightarrow v)$ -path OR a $(v \rightarrow u)$ -path exists

2.3 Paths, Walks, and Cycles

Concept	Definition
Walk	Sequence of vertices where consecutive vertices are connected — vertices and edges may repeat
Path	Walk with all vertices distinct
Cycle	Walk where first = last vertex, all others distinct
Semiwalk (digraph)	Each consecutive pair is connected in <i>either</i> direction
Semipath (digraph)	Semiwalk with all vertices distinct

Length of a path: Number of edges (unweighted) or sum of edge weights (weighted).

2.4 Cuts and Connectivity

Cuts (edge-based):

Term	Definition
Cut (edge-cut)	Set $C \subseteq E$ whose removal disconnects G
Cutset (edge-cutset)	Minimal cut — no strict subset of C is itself a cut
Bridge	Single edge whose removal disconnects G ; edge-connectivity = 1

Vertex cuts:

Term	Definition
Vertex-cut	Set $C \subseteq V$ whose removal disconnects G (or reduces it to a trivial graph)
Vertex-cutset	Minimal vertex-cut
Cut vertex	Single vertex whose removal disconnects G

Connectivity values:

Symbol	Name	Meaning
$\lambda(G)$	Edge-connectivity	Minimum cardinality of an edge-cutset
$\kappa(G)$	Vertex-connectivity	Minimum cardinality of a vertex-cutset

Key relationship: $\kappa(G) \leq \lambda(G)$ — vertex-connectivity $\leq$ edge-connectivity (always).

Network survivability: To survive one link failure $\rightarrow \lambda(G) \geq 2$. To survive one node failure $\rightarrow \kappa(G) \geq 2$. To survive two simultaneous link failures $\rightarrow \lambda(G) \geq 3$.

2.5 Trees and Spanning Trees

Term	Definition
Tree $T = (V, F)$	Connected graph with no cycles; always has exactly $ V - 1$ edges
Spanning tree	Subgraph of G that is a tree AND has the same vertex set as G

Key property of a tree: For every two vertices s, t , there is *exactly one* path between them → no ambiguity in routing; no cycles.

3. Minimum Spanning Tree

Problem Definition

Given a weighted graph, find a spanning tree with **minimum total edge weight**.

Characteristics of MST solutions:

- Connects all nodes with minimum cost infrastructure
- No fault-tolerance (single path between any pair of nodes)
- Unique routing (no cycles)

Applications:

- Minimum-cost cable/fiber topology (e.g., CATV fiber-to-the-street-corner)
 - Virtual private tree networks
 - Electricity networks, sewer systems
-

Kruskal's Algorithm

Idea: Greedily add the cheapest edge that does NOT create a cycle.

Steps:

1. Start with $P = \{\{v_1\}, \{v_2\}, \dots, \{v_n\}\}$ — each vertex in its own component
2. Sort all edges by weight (ascending)
3. For each edge (in order): if its two endpoints are in **different** components, add the edge to the tree F and **merge** the two components
4. Stop when all vertices are in one component (spanning tree is complete)

Example (Michigan road network):

Edges added in order:

edge 56 (weight 85) $\rightarrow P: \{1\}, \{2\}, \{3\}, \{4\}, \{5,6\}$

edge 34 (weight 105) $\rightarrow P: \{1\}, \{2\}, \{3,4\}, \{5,6\}$

edge 12 (weight 112) $\rightarrow P: \{1,2\}, \{3,4\}, \{5,6\}$

edge 25 (weight 135) $\rightarrow P: \{1,2,5,6\}, \{3,4\}$

edge 45 (weight 140) $\rightarrow P: \{1,2,3,4,5,6\}$ ✓ done

Key rule: An edge is rejected only if BOTH its endpoints are already in the same component (would create a cycle).

The algorithm is **correct** — provable by contradiction (proof by exchange argument).

4. Shortest Path Problem

4.1 Unweighted Graphs — Moore Algorithm (BFS)

Distance $d(u,v)$: Number of edges on the shortest $u \rightarrow v$ path (= hop count).

Moore Algorithm (Breadth-First Search):

1. Label source s with 0; all others with ∞
2. Label all neighbors of s with 1
3. For every unlabeled vertex adjacent to a vertex labeled k , assign label $k+1$
4. Continue until t is reached or no new labels can be assigned

Result: Every vertex v gets label $d(s,v)$. Works on both graphs and digraphs.

4.2 Weighted Graphs with Positive Weights — Dijkstra's Algorithm

Condition: ALL edge weights must be **strictly positive** ($c(e) \geq 0$).

Idea (label-setting): At each step, permanently fix the label of the vertex with the smallest temporary label.

Steps:

1. Label source s with (distance=0, predecessor= s); all others with (∞ , -)
2. Add s to the permanent set P
3. Update temporary labels of all neighbors of the newly added vertex
4. Add to P the vertex with the smallest temporary label $\rightarrow$ it gets a permanent label
5. Repeat steps 3–4 until t is reached (or all vertices are in P)

Example result (from slides): Starting from σ_0 , permanent order: $\sigma_0 \rightarrow v5 \rightarrow v2 \rightarrow v4 \rightarrow v3 \rightarrow$ ($v6, v7$ unreachable)

Real-world application — OSPF (Open Shortest Path First):

- Each router collects Link State Advertisements (LSAs) from all other routers
- Each router builds a complete map of the network topology
- Dijkstra is run at each router to compute the shortest path tree
- The resulting shortest path tree populates the router's routing table

4.3 Weighted Graphs with Negative Weights — Ford-Bellman-Moore (FBM)

Why needed: Dijkstra fails with negative weights. FBM is a **label-correcting** algorithm — all labels remain temporary until the end.

Condition: No **negative cycles** (a cycle whose total weight is negative $\rightarrow$ infinite loop).

Bellman's optimality conditions (labels $l(i)$ are optimal if AND only if):

1. $l(s) = 0$
2. $l(i)$ is the length of some actual (s,i) -path
3. $l(j) \leq l(i) + c(i,j)$ for every arc $(i,j) \in A$

Algorithm (pseudocode):

```

 $l(s) = 0$ ; all others  $l(v) = \infty$ 
Insert  $s$  into List
while List is not empty:
    remove first element  $v$  from List
    for each neighbor  $w$  of  $v$ :
        if  $l(w) > l(v) + c(v,w)$ :
             $l(w) = l(v) + c(v,w)$ 
            if  $w$  not in List: add  $w$  to back of List

```

Real-world application — Distance Vector Protocol (DVP):

- Each router maintains a distance vector (distances to all other nodes)
- Routers share distance vectors with neighbors only (not full topology)
- Vectors are updated iteratively — closely mirrors FBM
- Used in protocols like RIP (Routing Information Protocol)
- Contrast with OSPF (link-state, full topology, uses Dijkstra)

	Link State (OSPF)	Distance Vector (RIP)
Data shared	Full topology (LSAs)	Only distance vectors
Algorithm	Dijkstra	Ford-Bellman-Moore
Knowledge	Complete network map	Only neighbors

4.4 All-Pairs Shortest Path — Floyd's Algorithm

When to use: Need shortest paths between ALL pairs of vertices simultaneously.

Two approaches:

1. Run Dijkstra p times (once per source vertex): time $O(p \times p^2) = O(p^3)$
2. Floyd's algorithm: also $O(p^3)$ but simpler to implement

Condition: No negative cycles.

Floyd's Algorithm (pseudocode):

```
D(0) = adjacency matrix (0 on diagonal,  $\infty$  if no direct edge,  $c(e)$  otherwise)
for step = 1 to p:
  for all pairs (i, j):
    d[i][j] = min( d[i][j], d[i][v_step] + d[v_step][j] )
```

Interpretation of each step: "Would routing through vertex v_{step} make the path from i to j shorter?"

Example (5-node graph): After 5 steps (one per vertex), the matrix converges to all-pairs shortest distances.

⚠ All calculations must be done on the **matrix**, not by inspecting the graph picture — intermediate results may not correspond to valid single paths until the final iteration.

5. Maximum Flow Problem

5.1 Network Flow Model

A **network N** = weighted, capacitated digraph (V, A) where each arc a has:

- **Cost $c(a)$** — cost per unit of flow (used in min cost flow)
- **Capacity $u(a)$** — upper bound on flow

Flow f : $A \rightarrow \mathbb{R}^+$ subject to two constraints:

1. Flow conservation (Kirchhoff's law for networks):

$$\sum_{a \in I(v)} f(a) - \sum_{a \in I'(v)} f(a) = b(v), \quad \text{for every vertex } v$$

- $b(v) > 0 \rightarrow$ supply vertex (source)
- $b(v) < 0 \rightarrow$ demand vertex (sink)
- $b(v) = 0 \rightarrow$ transit vertex (flow in = flow out)

2. Capacity constraints:

$$0 \leq f(a) \leq u(a), \text{ for every arc } a$$

5.2 Residual Network Concept

Purpose: Track how much MORE flow can be pushed (or canceled) on each arc.

For each arc (i,j) in the original network carrying flow $f(i,j)$:

Arc in residual network	Residual capacity	Meaning
Forward arc (i,j)	$r(i,j) = u(i,j) - f(i,j)$	Extra capacity available
Backward arc (j,i)	$r(j,i) = f(i,j)$	Flow that can be canceled

Only arcs with **strictly positive** residual capacity are kept in the residual network.

Why it matters: Without the residual network, a greedy algorithm could commit to a bad flow early on and miss the optimal solution. Backward arcs allow "undoing" decisions.

5.3 Flow Augmenting Path Algorithm

Goal: Find the maximum flow from source s to sink t .

Augmenting Path: Any directed path from s to t in the **residual** network $N(f)$.

Residual capacity d of a path P : $d = \min \{ r(i,j) : (i,j) \in P \}$

Algorithm:

```
f = 0 (zero flow everywhere)
while N(f) contains an augmenting path from s to t:
    find any augmenting path P
    d = min residual capacity along P
    augment flow by d units along P
    update N(f)
```

Correctness (Augmenting Path Theorem): A flow f^* is maximal **if and only if** the residual network $N(f^*)$ contains **no** augmenting path from s to t .

Step-by-step example (1→4 max flow):

```
Initial: all capacities, zero flow
Step 1: Augment 4 units along path 1-3-4 → residual capacity min(4,5)=4
Step 2: Augment 1 unit along path 1-2-3-4 → residual capacity min(2,3,1)=1
Step 3: Augment 1 unit along path 1-2-4 → residual capacity min(1,1)=1
→ No more augmenting paths. Maximum flow = 6.
```

Applications:

- Maximum bandwidth from server to client
- Network restoration: finding spare capacity after a link failure
- Computing connectivity (see §5.5)

5.4 Max-Flow Min-Cut Theorem

s-t Cut: A partition of V into two sets S (containing s) and T (containing t). The cut capacity = sum of capacities of arcs from S to T .

Theorem (Max-Flow Min-Cut):

The **maximum value of an s-t flow** = the **capacity of the minimum s-t cut**.

Intuition: Any flow from s to t must cross every s - t cut, so the flow $\leq$ min-cut capacity. The theorem states these bounds are equal.

Consequence: To find the minimum cut $\rightarrow$ find the maximum flow. They solve each other.

5.5 Connectivity via Max Flow

Edge-Connectivity $\lambda(G)$

Key insight (from max-flow min-cut): The minimum number of edges to remove to disconnect v from w = the maximum number of **edge-disjoint paths** from v to w .

Method to compute $\lambda(v,w)$:

1. Replace each undirected edge $\{u,v\}$ with two anti-parallel arcs (u,v) and (v,u) , each with capacity 1
2. Run max-flow from v to w
3. $\lambda(v,w) = \text{max flow value } F(v,w)$

Edge-connectivity of G : $\lambda(G) = \min$ over all pairs (v,w) of $\lambda(v,w)$

Vertex-Connectivity $\kappa(G)$

Key insight: Minimum vertex cut between v and w = maximum number of **vertex-disjoint paths** from v to w .

Method (node splitting):

1. Split every vertex x into two copies: x' (in) and x'' (out)
2. Add arc $(x' \rightarrow x'')$ with capacity 1 for every original vertex x
3. For every original edge $\{u,v\}$: add arcs $(u'' \rightarrow v')$ and $(v'' \rightarrow u')$ with capacity 1
4. Run max-flow from v'' to w'
5. $\kappa(v,w) = \text{max flow value } F(v'', w')$

Vertex-connectivity of G : $\kappa(G) = \min$ over all non-adjacent pairs (v,w) of $\kappa(v,w)$

Survivability Requirements

Failure scenario	Required connectivity
Survive 1 link failure	$\lambda(G) \geq 2$

Failure scenario	Required connectivity
Survive 1 node failure	$\kappa(G) \geq 2$
Survive 2 simultaneous link failures	$\lambda(G) \geq 3$

6. Minimum Cost Flow Problem

6.1 Problem Definition

Goal: Send a given **demand d** from source s to sink t through a capacitated network at **minimum total cost**.

Optimization model:

$$\begin{aligned}
 &\text{minimize } \sum_{a \in A} c(a) \cdot f(a) && \text{(minimize total cost)} \\
 &\text{subject to:} \\
 &\quad \sum f(a) - \sum f(a) = b(v) && \text{(flow conservation)} \\
 &\quad 0 \leq f(a) \leq u(a) && \text{(capacity constraints)}
 \end{aligned}$$

where $b(s) = +d$, $b(t) = -d$, $b(v) = 0$ for all other vertices.

Relationship to other problems:

- **Shortest path:** special case of min cost flow with unlimited capacities ($u(a) = \infty$)
 - **Maximum flow:** special case with zero costs ($c(a) = 0$), maximize flow
 - Min cost flow is the **most general** of the three problems
-

6.2 Residual Network for Min Cost Flow

Same structure as for max flow, but now arcs have **costs** too:

Arc in residual	Residual capacity	Cost
Forward arc (i,j)	$r(i,j) = u(i,j) - f(i,j)$	$+c(i,j)$
Backward (artificial) arc (j,i)	$r(j,i) = f(i,j)$	$-c(i,j)$

Why $-c(i,j)$ for backward arcs: Pushing flow backward = canceling existing flow = saving cost
→ negative cost (reward).

⚠ Parallel arcs in the min-cost residual network **cannot be merged** (unlike in max-flow), because the parallel arcs have different costs.

6.3 Busacker-Gowen Algorithm (Successive Shortest Path)

Idea: Repeatedly augment flow along the **cheapest (shortest-cost)** augmenting path in the residual network until the demand is satisfied.

Algorithm:

```
f = 0; F = 0
while F < d:
    find the SHORTEST (min-cost) augmenting path P from s to t in N(f)
    δ = min(min residual capacity along P, d - F)
    augment f by δ along P
    update N(f)
    F = F + δ
```

Key detail: Because the residual network contains **negative-cost** artificial arcs, the shortest path must be found using the **Ford-Bellman-Moore** algorithm (not Dijkstra!).

Contrast with max-flow: Max-flow picks *any* augmenting path; Busacker-Gowen picks the *cheapest* one.

Step-by-step example (demand $d = 4$):

Network: $s \rightarrow v_4 \rightarrow v_3 \rightarrow v_1 \rightarrow v_2 \rightarrow t$ and $s \rightarrow v_1 \rightarrow v_2 \rightarrow t$, etc.

Iteration 1:

Shortest path in $N(\emptyset)$: $s-v_4-v_3-v_1-v_2-t$, cost=5, capacity=2

Augment 2 units, cost contribution = 10

Iteration 2:

Shortest path in $N(f)$: $s-v_1-v_3-t$, cost=7, capacity=2

Augment 2 units $\rightarrow F=4=d \checkmark$, cost contribution = 14

Total cost = 10 + 14 = 24

Paths used: 2 units on $s-v_1-v_2-t$, 2 units on $s-v_4-v_3-t$

6.4 Multiple Supply/Demand Vertices

Generalization: Multiple sources $s_1, s_2, \dots$ with supplies $b(s_i) > 0$ and multiple sinks $t_1, t_2, \dots$ with demands $b(t_j) < 0$.

Solution method — super-source/super-sink trick:

1. Add a **super-source** σ with zero-cost arcs to each supply vertex s_i , capacity = $b(s_i)$
2. Add a **super-sink** τ with zero-cost arcs from each demand vertex t_j , capacity = $-b(t_j)$
3. Run Busacker-Gowen from σ to τ with total demand $d = \sum b(s_i)$

This reduces the multi-source/multi-sink problem to a single-source/single-sink one.

7. Multi-Commodity Flow Problems

Single vs Multi-Commodity

	Single-commodity	Multi-commodity
Definition	All units of flow are	Different flows have different source-

	Single-commodity	Multi-commodity
	identical	destination pairs
Example	Oil pipeline: all oil is equivalent	Phone network: each call is a distinct commodity
Integer solutions	Always exist for integer capacities	NOT guaranteed — fractional solutions may be cheaper
Algorithms	Efficient graph algorithms exist	No efficient general graph algorithm known

Multi-Commodity Min Cost Flow Problem

Decision variable: $f_k(a)$ = flow of commodity k on arc a

Objective: Minimize total cost $\sum_a c(a) \cdot \sum_k f_k(a)$

Constraints:

- Flow conservation: per commodity individually
- Capacity: **sum** of all commodity flows on an arc $\leq$ arc capacity

Integer vs Real Relaxation

For multi-commodity problems, the integer and real formulations can give **different optimal solutions**. Example (triangle, all capacities = 1, 3 commodities f_1, f_2, f_3):

- Real (linear) optimal cost = 153
- Integer optimal cost = 202

For many applications (optical networks with whole wavelengths), integer solutions are required — but integer multi-commodity flow is NP-hard.

Application: IP Routing vs Traffic Engineering

	Pure IP (OSPF)	Traffic Engineering (MPLS/TE)
Route choice	Shortest path only	Explicit arbitrary routes
Algorithm	Dijkstra	Multi-commodity min cost flow
Congestion avoidance	No — all traffic follows shortest path	Yes — traffic spread across multiple paths
Problem type	Single-commodity (all flows share same shortest path)	Multi-commodity (each OD pair is a commodity)

IP packet granularity (a few bytes) vs link capacity (100s Mbit/s) → flow variables are essentially continuous → real-valued formulation is a good approximation.

Limitations of Graph Algorithms

Graph algorithms handle "easy" (polynomial-time solvable) problems efficiently. Complex problems like network design remain beyond simple graph algorithms → Chapter 4 introduces general optimization techniques (LP, ILP) to fill this gap.

8. Network Availability

8.1 Definitions

Term	Symbol	Meaning
Time Between Failures	TBF	Time interval between consecutive failures
Time To Repair	TTR	Duration of a repair/outage
Mean Time Between Failures	MTBF	Average TBF
Mean Time To Repair	MTTR	Average TTR

Term	Symbol	Meaning
Availability	A	Fraction of time a component is working
Unavailability	$U = 1 - A$	Fraction of time a component is down

$$A = \frac{MTBF - MTTR}{MTBF} \approx \frac{MTBF}{MTBF + MTTR}$$

Typical component values (1 year $\approx 10^4$ hours):

Equipment type	MTBF (hours)	MTTR (hours)
Web server	$10^4 - 10^6$	1
IP interface card	$10^4 - 10^5$	2
IP router	$10^4 - 10^6$	2
SDH switch	$10^5 - 10^6$	4
1 km cable	$10^6 - 10^7$	48
GSM base station	$10^4 - 10^6$	1–4
MANET node	< 10	n/a

8.2 Availability of Combinations

Serial Connection (AND logic — both must work)

For two elements in series:

$$A_{tot} = A_1 \cdot A_2$$

General (n elements in series):

$$A_{tot} = \prod_i A_i$$

Worst element dominates.

Parallel Connection (OR logic — at least one must work)

For two elements in parallel:

$$A_{tot} = 1 - U_1 \cdot U_2 = 1 - (1 - A_1)(1 - A_2) = A_1 + A_2 - A_1 A_2$$

General (n elements in parallel):

$$U_{tot} = \prod_i U_i$$

Best element dominates.

8.3 Availability of a Network Path

For a path using links $l_1, l_2, \dots$ and nodes $k_1, k_2, \dots$:

$$A_{path} = \prod_{k \in path} A_k \cdot \prod_{l \in path} A_l$$

Protected connection (working path + recovery path): The connection is available if either the working path OR the recovery path works → parallel combination of two serial paths.

8.4 Most Available Path Problem

Goal: Find the path from s to t that maximizes end-to-end availability.

Key transformation — converting to shortest path:

$$\max_p A_p = \max_p \prod_{i \in p} A_i$$

Taking the negative logarithm (ln is monotone, negative flips max→min, product→sum):

$$= \min_p \sum_{i \in p} (-\ln A_i)$$

This is exactly the shortest path problem with link/node weights $-\ln(A_i) \geq 0$!

Since all availabilities $A_i \in (0, 1]$, we have $-\ln(A_i) \geq 0$, so **Dijkstra's algorithm** applies.

Comparison table:

	Shortest Path	Most Available Path
Weights	Link lengths/costs (≥ 0)	$-\ln(\text{availability})$ (≥ 0)
Objective	Minimize total weight	Minimize total weight
Algorithm	Dijkstra	Dijkstra
Optimality	Sum of weights minimal	Product of availabilities maximal

8.5 Protected Connection Availability

A **protected connection** has:

- A **working path** (primary)
- A **recovery path** (backup, via spare capacity)

The connection is available if **at least one** path is operational:

$$A_{\text{connection}} = 1 - (1 - A_{\text{working}}) \cdot (1 - A_{\text{recovery}})$$

For this to be computed, first calculate each path's availability (product of component availabilities along that path).

9. Algorithm Comparison Summary

Algorithm	Problem	Condition	Complexity	Key idea
Kruskal	Min spanning tree	Undirected, weighted	$O(q \log q)$	Greedy add cheapest non-cycle edge

Algorithm	Problem	Condition	Complexity	Key idea
Moore (BFS)	Shortest path	Unweighted	$O(p + q)$	Breadth-first label propagation
Dijkstra	Shortest path	Weights ≥ 0	$O(p^2)$	Label-setting: permanently fix cheapest temp. label
Ford-Bellman-Moore	Shortest path	Negative weights OK (no neg. cycles)	$O(p \cdot q)$	Label-correcting: update queue until stable
Floyd	All-pairs shortest path	No negative cycles	$O(p^3)$	Dynamic programming over intermediate nodes
Flow Augmenting Path	Maximum flow	—	$O(F \cdot q)$	Find and saturate augmenting paths in residual graph
Busacker-Gowen	Min cost flow	—	Iterative	Find CHEAPEST augmenting path (uses FBM as subprocedure)

Problem hierarchy (most general to most specific):

Min Cost Flow

- └ Shortest Path ($u(a) = \infty$, find cheapest route for 1 unit)
- └ Maximum Flow ($c(a) = 0$, maximize total flow)

10. Exercise Problems Overview

Exercise 1 — Degree vs Connectivity

- (a) If all node degrees $\geq n$, does this imply $\lambda(G) \geq n$ and $\kappa(G) \geq n$?
→ **No** in general. Degree is a local property; connectivity is global. Counterexample: two cliques connected by a single edge (bridge).
- (b) Construct graphs where degree = connectivity = 2, 3, and 4.

Exercise 2 — Routing Metrics in MANET

MANET = Mobile Ad-Hoc Network; nodes: delays in ms, links: energy in Joules. Route from A to B.

- (a) **Minimize total energy E** → weight links by energy → Dijkstra
- (b) **Minimize delay D** → weight nodes by delay → Dijkstra (with node-weighted shortest path)
- (c) **Minimize E subject to D < max** → Constrained shortest path → Min cost flow (Busacker-Gowen) with capacity limit modeling delay constraint
- (d) **Minimize E + D equally** → Combine weights (normalize and sum) → Dijkstra on combined weight

Exercise 3 — Fiber Network Design (CATV)

- Minimize total **cable length** to connect all street corners from head-end A → **Minimum Spanning Tree (Kruskal)**
- Note: cable length $\neq$ fiber length. MST minimizes cable; minimizing fiber requires shortest-path tree from the head-end.
- MST result is **independent of head-end location** (same edges); but total fiber length **depends** on the head-end.

Exercise 4 — Wireless Sensor Network

- (a) Determine network topology → Min spanning tree or connectivity analysis
- (b) Minimize power delivery from all sensors to internet gateway → **Multi-commodity min cost flow** (each sensor = one commodity)

- (c) Apply Busacker-Gowen on concrete numerical example
- (d) Load balancing between base stations → add capacity constraint on the super-sink arcs

Exercise 5 — Routing with Link AND Node Capacities

- General problem: both links AND nodes have capacities and costs
- **Solution:** Split each node v into v' and v'' with arc $(v' \rightarrow v'')$ having the node's capacity and cost → reduces to standard min cost flow → Busacker-Gowen
- Particular problem ($F=6$ Mbit/s): 3 paths of 2 Mbit/s each, total cost = 72

Exercise 6 — PC-Server Connections

- 4 PCs, 2 servers, each PC needs 20 Mbit/s; minimize total connection cost
- Single-commodity min cost flow with multiple supply (servers) and demand (PCs) vertices → super-source/super-sink extension of Busacker-Gowen
- Result: paths of cost 10, 3, 3, 4 → total cost = 20

Availability Exercises

Exercise 1 (Duplicated path — 9-node network):

- (a) A-B-C-D-E availability = product of all components along path
- (b) A-F-I-E availability = product along that path
- (c) Protected A-E = $1 - (1 - A_{\text{path1}}) \times (1 - A_{\text{path2}})$

Exercise 2 (MANET availability):

- Find most reliable path from F to E → transform: assign weight $-\ln(A_i)$ to each link/node → Dijkstra
- Disruption probability = $1 - A_{\text{path}}$

Exercise 3 (Component availability influence):

- Halving failure probability → compute new availability from formula
 - Duplicating components → parallel connection formula
-

Sources: Pickavet, M. (2025). MOBAN Chapter 3 slides — Fundamental Network Modelling Techniques. IDLab, Ghent University.